

0.8.4. #include <reste> — a % b

#include <reste> : $a \% b$, le reste d'une division, demande une routine, sauf par 1, 2, 4, 8, 16... écrits en clair.

1. Ce qu'il faut comprendre

Ce que c'est : $a \% b$ est le **reste** de la division : $100 \% 7$ vaut 2, parce que $100 = 7 \times 14 + 2$. Par 2, 4, 8, 16... écrit en clair, c'est garder les bits du bas : gratuit. Sinon, il faut une routine.

La ligne à écrire : `#include <reste>`.

Ce qu'elle coûte : une vingtaine d'octets.

Essaie : `hasard() % 6` : un nombre de 0 à 5, pour un dé.

On retrouve reste dans 42 leçons

[0.9](#), [0.9.1](#), [0.9.2](#), [0.15](#), [0.15.1](#), [0.15.2](#), [0.77](#), [0.81.1](#), [0.81.3](#), [0.83](#), [0.85](#), [0.88.2](#), [0.89.2](#), [0.90.5](#), [0.91.4](#), [0.91.5](#), [0.91.6](#), [0.91.7](#), [0.91.8](#), [0.91.9](#), [0.92](#), [0.92.1](#), [0.92.2](#), [0.94](#), [0.94.1](#), [0.102](#), [0.102.1](#), [0.103](#), [0.104](#), [0.108](#), [10](#), [16](#), [31](#), [32](#), [60](#), [62](#), [94](#), [105](#), [111](#), [114](#), [115](#), [131](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **399 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      60 o ← la console, à l'allumage — la mise en route de la console
main()         27 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireNombre — attend de pouvoir écrire une case
EcrireNombre   33 o ← nombre() — écrit un nombre en chiffres
Reste          10 o ← main() — le reste d'une division
CopierTuiles   31 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles        160 o ← CopierTuiles — 10 lettres sur 43
```

Lettres gravées : 0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
nombre() × 1    10 o sur place, dans main() + EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <reste> : a % b ----  
  
#include <reste>      // la routine du reste  
#include <nombre>
```

2. UNE VARIABLE — A

```
uint8_t a = 100;
```

3. UNE VARIABLE — B

```
uint8_t b = 7;
```

4. UNE VARIABLE — R

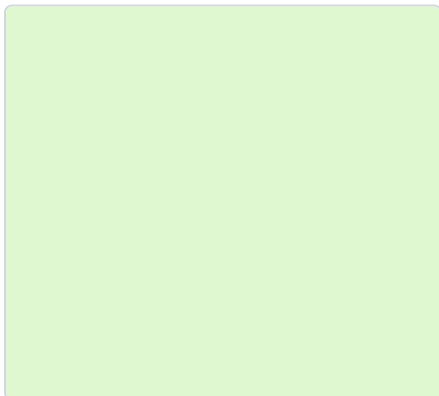
```
uint8_t r = 0;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    r = a % b;          // 100 = 7 × 14 + 2 : le reste est 2  
    nombre(8, 8, r);    // 002  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

002 au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ `100 % 7 = 2`

✓ `002` s'affiche

Ensuite : **0.8.5. `#include <diviser>` — `a / b`** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++